Drift Detection, Diagnosis and Reliability Analysis for Deployed Machine Learning Models

Rahul Guhagarkar[1*], Nathan Cherian[1], Alok Pal[1], Akul Patre[1], Sanjay Mirchandani[1]

[1] Department of Computer Engineering, Vivekanand Education Society's Institute of Technology, Hashu Advani Memorial Complex, Collector's Colony, Chembur, Mumbai 400074, India

[*] Corresponding author, e-mail: 2023.rahul.guhagarkar@ves.ac.in

Abstract

Deployed machine learning models lose accuracy when production data drift away from the training data. Monitoring tools raise an alarm when this happens but seldom say what kind of drift occurred, although the right response depends on it: covariate shift calls for reweighting, prior shift for recalibration, concept drift for fresh labels and label noise for a check of the labeling process. This paper presents Data Drift Autopsy, an end-to-end monitoring pipeline that detects drift, estimates the lost accuracy without labels, locates the shifted features, names the drift type, scores the trust due to each prediction and advises whether a cheap retrain will help. At its heart sits a diagnosis made only of hypothesis tests, all held to one family-wise error rate. Its first stage uses no labels and tells covariate shift from prior shift. The second stage is new: given a few hundred late labels, it asks whether the errors have moved in a way the model's own confidence cannot account for, the mark that sets concept drift apart from label noise. With drift injected into two census tasks and two model families, the diagnosis named the type in 85.7 to 94.3% of windows, ahead of common monitoring rules. A learned meta-classifier did better on drift it had seen, yet fell to 31.1 to 51.2% on held-out drift mechanisms, where our tests still reached 62.4 to 81.5%. On four real datasets with documented changes, the pipeline flagged no control window and pointed to the known events.

Keywords

concept drift, label shift, model monitoring, drift diagnosis, hypothesis testing

1 Introduction

Nobody announces it when a deployed classifier starts to slip. The production data simply wander away from the training data, while the labels that would expose the loss tend to turn up weeks or months later [1]. A monitor has to catch the change and make sense of it before those labels exist.

Existing monitors do the first part of that job well. They flag a changed distribution [1] and give a fair estimate of how much accuracy has gone [2, 3]. Far fewer say what kind of change took place. That gap matters, since each kind has its own remedy. Moreno-Torres et al. [4] distinguish three types of dataset shift. Under covariate shift the inputs drift but the mapping from inputs to labels holds, so reweighting the training data is often enough. Under prior shift it is only the class balance that moves, which recalibration usually fixes. Concept drift alters the mapping itself; nothing short of fresh labels will repair it. To these we add label noise as a fourth type. Here the labels used to grade the model are themselves corrupted, so the fault lies in how labels are collected rather than in the model.

Telling these types apart is harder than detecting that something changed. Covariate shift and prior shift both move the inputs and the predictions. Concept drift and label noise, on the other hand, leave the inputs as they were, so no test that looks only at unlabeled data can see them.

This paper presents Data Drift Autopsy, an end-to-end monitoring pipeline that takes a deployed model, its labeled training data and a stream of unlabeled production data. For every production window it answers the questions a monitoring team asks in turn. Has the data changed? How much accuracy has been lost? Which features moved? What type of drift is it and what probably caused it? Which individual predictions should not be trusted? Would a cheap retrain recover the loss? The pipeline works for tabular and image models. Every result is written to one report that an interactive dashboard presents.

The core of the pipeline is the step that names the drift type, which is also the main subject of this paper. Nothing in this step is learned from examples of drift: it is a handful of hypothesis tests that share one significance level and one family-wise error rate. The first stage, which needs no labels, combines existing tools. After testing the inputs and the predictions for a shift, it estimates the new class balance with black-box shift estimation (BBSE) [5]

and, borrowing an idea from Roschewitz et al. [6], checks whether that balance alone accounts for the change in the inputs. If it does, the shift is a prior shift; if it does not, it is covariate shift. The second stage is new. It uses a few hundred late labels to ask whether accuracy has dropped and, if so, whether the errors changed in a way that depends on some input feature. Uniform label noise changes the error rate only in line with how confident the model is, while concept drift changes it inside one region of the input space. Because every step is a formal test, the false alarm rate is fixed before the data are seen.

We evaluate the diagnosis on a benchmark of injected drift where the true type is known. We also run the whole pipeline on four real datasets whose changes are documented. The benchmark includes drift mechanisms that the tests were not designed for, so that a method tuned to its own benchmark cannot look good by construction. The contributions are as follows.

1 An audit test that separates concept drift from label noise using only a few hundred late labels. Existing shift tests, such as DetectShift [7], either need labeled production data or do not consider label noise.
2 A hierarchical procedure that combines this audit with a label-free stage and names four drift types under family-wise error control.
3 A benchmark covering two tabular tasks and two model families, with matched as well as held-out drift mechanisms. It shows that tests built on the definitions of the types carry over to new drift, while a learned rule does not.
4 An end-to-end monitoring pipeline that covers detection, accuracy estimation, root cause analysis, diagnosis, reliability scoring and remediation triage, evaluated on four real drifts with documented causes.

Section 2 positions the work against existing methods. Sections 3 to 5 present the pipeline, the experiments and the results, while Section 6 discusses them and concludes.

2 Related work

Work related to ours comes from several directions: how drift types are defined, how drift is detected, how label shift is estimated and how accuracy can be estimated without labels. Before reviewing each of them, Table 1 sets our work beside the methods closest to it. These are black-box shift detection (BBSD), BBSE, confidence-based performance estimation (CBPE), DetectShift and the method of Roschewitz et al.

Table 1 Capabilities of the closest methods and of ours. A tick marks a capability that the method provides

Method	Error rate stated	Accuracy estimate	Localizes features	Types named	Concept vs noise	Held-out drift
BBSD [1]	√	–	limited	–	–	–
BBSE [5]	√	–	–	prior	–	–
CBPE [3]	–	√	–	–	–	–
DetectShift [7]	√	–	–	needs labels	–	–
Roschewitz et al. [6]	√	–	–	prior, cov.	–	–
Ours	√	√	√	four	√	√

Each existing method covers one or two of the columns. Roschewitz et al. come closest, separating prior from covariate shift without labels, while DetectShift names types only when the production data are labeled. The paragraphs below look at each line of work in turn.

Drift types. Moreno-Torres et al. [4] define covariate shift, prior shift and concept shift by the part of the joint distribution of inputs and labels that changes. DetectShift tests for a change in this joint distribution and in each of its parts, $P(X)$, $P(Y)$, $P(X \mid Y)$ and $P(Y \mid X)$ [7]. It needs labels for the new data, however, which its authors acknowledge as a limitation. We keep the same definitions, add label noise as a fourth type and ask for labels only in a small audit.

Drift detection. Some detectors follow the error rate of the model, for example the drift detection method (DDM) [8]. They need a label for every prediction and can detect concept drift, but they cannot tell it apart from label noise. Label-free detectors compare the inputs or the model outputs instead. Rabanser et al. [1] compared univariate tests, kernel tests [9], classifier two-sample tests [10] and BBSD on the model outputs. BBSD worked best overall. Each of these tests says that a shift happened, not what type it was. Our pipeline uses such tests as building blocks and goes on to name the type.

Label shift. BBSE recovers the new class prior from the confusion matrix of any classifier [5]. It assumes that the inputs inside each class stay as they were, which is exactly the property that separates prior shift from covariate shift. Roschewitz et al. [6] test this property on medical images. They resample the reference data to the estimated prevalence and attribute the shift to prevalence if the features then stop differing. Our label-free stage follows the same idea. It differs in two ways: it uses importance weights in place of resampling and it puts every label-free decision into one family with a stated error rate.

Performance estimation without labels. Average confidence (AC) [11], difference of confidences (DoC) [12], average thresholded confidence (ATC) [2] and CBPE [3] all estimate accuracy from what the model outputs. Each assumes that the link between confidence and accuracy holds in production. Models often stay confident while their accuracy falls under shift [13], so the assumption can fail. We report CBPE next to the other three and read it as an upper bound when the assumption breaks.

Explanation, reliability and benchmarks. Shapley additive explanations (SHAP) [14] attribute a prediction to the input features. Class activation maps (CAM) [15] do the same for image regions. Out-of-distribution (OOD) scores such as the Isolation Forest [16] flag unusual inputs, while importance weighting corrects covariate shift [17–19]. Folktables [20] and the Continual LEArning on Real-World Imagery (CLEAR) benchmark [21] provide real shifts, although nobody knows their exact type. Injected shifts give a ground truth [1] but may favor the method being tested. Each of these tools serves one stage of our pipeline. Our benchmark holds out drift mechanisms to limit the risk of favoring the tests.

In short, each prior method solves part of the problem. To our knowledge, no published method separates concept drift from label noise with a small label audit. Neither has any type diagnosis been tested on drift mechanisms held out from its design.

3 Method

This section first defines the four drift types precisely and then follows the data through the pipeline introduced in Section 1. Its input is a deployed model, the labeled data on which it was built and a stream of unlabeled production data, plus a few hundred late labels once they arrive. Fig. 1 lays out the stages.

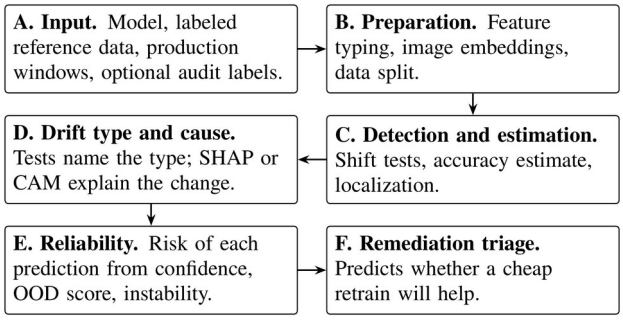


Fig. 1 Stages of the monitoring pipeline, from the inputs (A) to the advice on retraining (F)

Stages A to C bring the data into a common form, test whether anything has changed and estimate how much accuracy the change may cost. Stage D, the core of the paper, names the type of change and points to its likely cause. Stages E and F then act on what was found, first for single predictions and then for the decision to retrain. Only the optional audit in stage D uses production labels. With one exception, every decision in the pipeline is a hypothesis test at the significance level $\alpha = 0.05$. The exception is the population stability index (PSI), which raises an alarm at a conventional threshold. When many tests are run together, the chance of at least one false alarm is held at α by the Holm procedure [22], its weighted form or its Bonferroni bound.

3.1 Notation and drift types

To test for a drift type we first have to say exactly what that type changes. A little notation makes this easier. Write $P(X, Y)$ for the distribution of inputs X and labels Y behind the reference data. A production window (a batch of rows from a later period or from another region) is drawn from $Q(X, Y)$. On every row the model outputs a predicted label $\hat{y}$ together with a confidence c, taken as its largest class probability. The four types differ in what changes between P and Q. Covariate shift has $Q(X) \neq P(X)$ while $Q(Y \mid X) = P(Y \mid X)$. Prior shift has $Q(Y) \neq P(Y)$ while $Q(X \mid Y) = P(X \mid Y)$. Concept drift has $Q(Y \mid X) \neq P(Y \mid X)$. Label noise is different again: the true distribution does not move at all, yet each label we observe may have been flipped with some probability. Two more terms recur below. An audit means a random sample of m production rows whose labels come in late. A control window is drawn from the reference distribution itself, so no alarm should fire on it. In practice a real window may carry several types at once; Section 6 comes back to this.

3.2 Reference split, detection and estimation

Stage C answers three questions: have the data moved, how much accuracy might the change cost and where in the inputs is it located? All three need a clean reference to compare against. The labeled reference data are therefore split into three disjoint parts, stratified by class: 60% for training, 20% for calibration and 20% for testing. The model is fitted on the training part alone. The calibration part is the reference window for every detector and fits the accuracy estimator. The test part gives the reference accuracy and doubles as the control window. Nominal features, such as an occupation code, are treated as

categories because their numeric order means nothing. Images are mapped to the 512-dimensional output of the global average pooling layer of a frozen residual network with 18 layers (ResNet-18) trained on ImageNet [23]. Image drift can then be tested with the same tests as tabular drift [1].

Six detectors decide whether the inputs or the model outputs have moved. The Kolmogorov–Smirnov (KS) test compares each numeric feature, with a Bonferroni correction over features. PSI raises an alarm at its conventional value of 0.2 [24]. The maximum mean discrepancy (MMD) test [9] compares all standardized inputs jointly through a Gaussian kernel, with a permutation p value. The confidence test compares histograms of c with a χ^2 test of homogeneity and pools sparse cells until every expected count reaches 5 [25]. Two further detectors from Rabanser et al. [1] act as baselines. BBSD applies a KS test to each class probability. The domain classifier trains gradient boosted trees to tell 3,000 reference rows from as many production rows. An exact binomial test then compares its held-out accuracy with $1/2$ [10]. Every permutation p value is given by $(1+b)/(1+B)$, where b of the B permutations reach the observed statistic [26].

Labels arrive late, so the accuracy is first estimated from the model's own confidence with CBPE [3]. A map $a(\cdot)$ from confidence to the probability of a correct prediction is fitted by histogram binning on the calibration part. For a window with confidences c_1 to c_n, the estimate $\hat{A}$ is given by

$$\hat{A}=\frac{1}{n}\sum_{i=1}^{n} a(c_i). \tag{1}$$

A confidence bin that holds fewer than 5 calibration rows cannot give a reliable accuracy, so predictions in such a bin keep their own confidence in Eq. (1). As baselines, AC, DoC and ATC (with the negative entropy score) are fitted on the same calibration part [2, 11, 12].

An alarm helps more when it also says where the change sits. Localization tests every feature, with KS for numeric features and the χ^2 homogeneity test for nominal ones, under Holm correction. It reports an effect size between 0 and 1 for each feature: the KS statistic or Cramér's V. For nominal features it also gives the share of production rows whose category never occurs in the reference.

3.3 Drift type diagnosis

Stage D turns an alarm into one of the four types and thereby into a response. It follows the decision path in Fig. 2. The questions in the upper row use no labels and separate covariate shift from prior shift. The lower row separates concept drift from label noise. It runs only when no label-free test rejects and an audit is available.

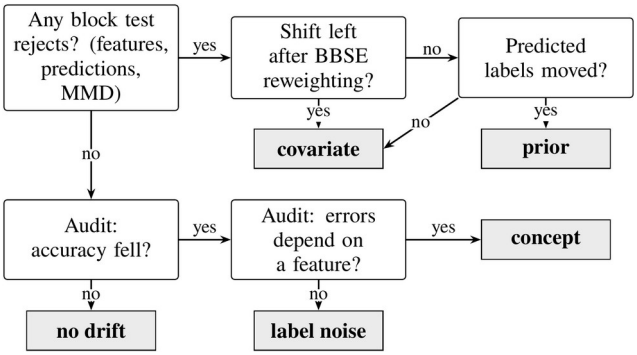


Fig. 2 Decision path that names the drift type. Questions are shown in white boxes and verdicts in gray boxes; the upper row needs no labels and the lower row uses the audit

Label-free stage. The first question is whether anything has moved at all. Three blocks of tests answer it: the feature tests, a χ^2 test on the predicted labels and the MMD test. They form one family tested at level α with a weighted Holm procedure [22]. The weight starts out split evenly, one third per block. Inside the feature block it is divided again among the individual features. A test that rejects hands its weight on to those still waiting, which is how the family-wise error rate survives every label-free decision. In its first step the procedure reduces to ordinary Holm over the three blocks, each feature block entering through its Bonferroni p value. We pool in this way on purpose. A permutation test cannot return a p value below $1/(B+1)$, so without pooling the MMD test would have little chance against dozens of feature tests. If nothing rejects, the stage stops and reports no shift.

A rejection raises the next question: could a change in the class balance alone produce what we see? To find out, BBSE first estimates the new class prior. Let $C_{jk}=P(\hat{y}=j, y=k)$ on the calibration part and let μ_j be the share of production predictions that fall in class j. The class weights $w_k=Q(Y=k)/P(Y=k)$ are given by the solution of

$$C\,w=\mu. \tag{2}$$

If the shift really is a pure prior shift, each class keeps its own input distribution. Reweighting every reference row by the weight w_y of its class, as given by Eq. (2), ought

then to reproduce the production inputs. Checking this is the label shift consistency test, which borrows the prevalence adjustment idea of Roschewitz et al. [6]. The feature tests are repeated against the weighted reference, using Kish's effective sample size [27] and Holm correction. A feature that still differs carries a covariate shift. If no feature differs and the predicted labels have moved, the shift is a prior shift. A prior shift always moves the predicted labels of an informative classifier, so a shift that leaves them in place is called covariate.

Audit stage. Concept drift and label noise do not touch the inputs or the model outputs, so only labels can reveal them. The audit draws m late labels and first checks, with a one-sided two-proportion test, whether accuracy has dropped. If it has, the next question is whether the change in errors depends on a feature. To answer it, a logistic model of whether a prediction is correct is fitted on the calibration rows and the audit rows together. Its null form holds 5 confidence bins at reference quantiles, bins of a feature f, an audit indicator and the interaction of that indicator with the confidence bins. The alternative adds the interaction of the indicator with the bins of f. A likelihood ratio test compares the two for each feature. A nominal category gets its own bin only if it occurs at least n_{min} times in the audit, where n_{min} is given by $\lceil 5/min(e, 1-e) \rceil$ and e is the audit error rate. The remaining categories share one bin.

The confidence term is what separates the two types. In a binary task, uniform label noise at rate r makes the probability of a correct prediction equal to $r+(1-2r)q(x)$, where $q(x)$ is the model's own probability of being right at x. The change therefore depends on x only through $q(x)$, which the confidence bins of the null absorb. Concept drift, by contrast, moves the errors inside a region of some feature. A feature that stays significant after Holm correction therefore points to concept drift. If no feature is significant, the verdict is label noise. Each of the two stages is tested at level α, so a window without drift receives a drift verdict with probability at most 2α.

3.4 Root cause analysis

The drift type suggests what kind of fix to consider, but not where the trouble started. Root cause analysis fills that gap by asking which inputs the model now leans on more, or less, than before. The answer narrows the search for a cause without proving one. For tabular models we compute SHAP values [14] of the deployed model with its preprocessing and compare the mean absolute attribution of every feature on reference and production rows, each change with a percentile bootstrap interval Bonferroni adjusted over the features. For image models, CAM [15] highlights the image regions behind the prediction of the linear head.

3.5 Reliability and remediation triage

Everything so far speaks about a window as a whole. A monitoring team usually wants two more things from it: to know which individual predictions to doubt and whether a cheap retrain would win the accuracy back. For single predictions we combine three signals, each scaled to lie between 0 and 1: the confidence risk $1-c$, the instability of the prediction under small perturbations of the input and an OOD score that blends an Isolation Forest [16] with a standardized distance. A logistic regression on the calibration part maps the three signals to a probability of error. We fit it only if the calibration part contains at least ten wrong predictions per signal [28]. With fewer, the risk falls back to $1-c$.

The triage, for its part, has to judge without production labels whether retraining is worth the trouble. It does so with two probes. The first relies on density ratio weights, which come from a classifier trained to tell reference rows from production rows; the weights are fitted on separate folds and capped at 20 times their mean [19]. A model retrained with them is set against an unweighted model on a held-out 30% of the reference, weighted in the same way [17, 18]. The second probe needs late labels from an earlier window. Where they exist, it retrains on them and checks the result on a held-out part of that same window. A retrain is recommended only if some probe predicts a gain that stays significant after Holm correction across the probes.

4 Experimental setup

The experiments are built around two questions. First, does the procedure name the type correctly when the truth is known, including for drift it was not designed for? Second, does it behave sensibly on real drift, where the type is unknown but the causes are on record?

Protocol. Every setting below was fixed before the evaluation that uses it. The diagnosis was frozen before the held-out design B was run for the first time. A later review of the code found two errors in the diagnosis. The label-free stage had used a gatekeeping rule whose family-wise error rate reached 6.6% at a nominal 5% in a simulation, against 4.9% for the weighted Holm procedure of Section

3.3. The audit had also counted nominal categories over the reference rows and audit rows together. Both errors were fixed without looking at the results and every result was produced again, which changed the verdict in 115 of the 5,300 benchmark windows. Permutation tests use 200 permutations. SHAP explains 1,000 rows per side against 100 background rows. Each real audit uses 600 labels from its own random stream. All randomness is seeded, so every result can be repeated exactly. Everything ran on one workstation with a 20-core processor and a graphics processor, using scikit-learn [29], SciPy, NumPy, SHAP, OpenCLIP and PyTorch.

4.1 Models

A diagnosis that worked for only one kind of model would be of limited use, so the pipeline monitors two untuned model families. The first is logistic regression (LR) with L_2 regularization ($C=1$) on standardized numeric and one-hot encoded nominal features. The second is histogram gradient boosting (GB) [30] from scikit-learn [29] with default settings, no early stopping and native splits on nominal codes. Codes beyond the 254 most frequent are pooled, which is the limit of the implementation. For CLEAR-10 both models work on the frozen embeddings. SHAP is reported for LR only, since for GB the exact explainer would take hours per window.

4.2 Datasets

The real datasets were chosen because the cause of their drift is documented, which lets us check the verdicts against known events. Table 2 lists the four of them. Two are census tasks from the American Community Survey (ACS) with data from California (CA), Texas (TX), New York (NY), Florida (FL) and Washington (WA). The other two are a gas sensor array and an image stream.

Table 2 Datasets, drift windows and reference accuracy

						Ref. accuracy	
Dataset	Reference	Windows	Rows	Inputs	Classes	LR	GB
ACS Income	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	36,162–203,278	10	2	0.826	0.833
ACS Employment	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	56,576–326,241	16	2	0.729	0.772
Gas sensor	batches 1–2	batches 3–10	161–3,613	128	6	0.982	0.985
CLEAR-10	bucket 1	buckets 2–10	3,297–3,300	512	11	0.914	0.897

Rows gives the smallest and the largest window, Inputs the number of features or embedding dimensions and the last two columns the reference test accuracy of LR and GB.

ACS Income and ACS Employment. Both tasks come from the ACS of the United States Census Bureau, as packaged by Folktables [20]. ACS Income predicts from 10 features whether a person earns more than $50,000. ACS Employment predicts from 16 features whether a person aged 16 to 90 is employed. CA 2014 is the reference. The temporal windows are CA 2015 to 2019 and 2021 to 2023, as no standard 2020 release exists [31]. The geographic windows are TX, NY, FL and WA in 2014. In 2019 the relationship variable RELP was replaced by RELSHIPP [32]. We map it back to RELP with the code labels of the 2018 and 2019 data dictionaries (Table S2 in the Supplement). Five documented changes fall in these years: new occupation codes in 2018 [33], a redesigned relationship question in 2019, a new coding of race answers from 2020 [34], the pandemic and wage growth against the fixed income threshold.

Gas sensor array drift. This dataset from the machine learning repository of the University of California, Irvine (UCI) holds 13,910 measurements of six gases by 16 metal oxide sensors, taken over 36 months in ten batches [35, 36]. Each measurement has eight features per sensor. As the sensors age, their response to the same gas drifts while the labels stay exact. This is a documented change in $P(X \mid Y)$. Batches 1 and 2 form the reference and batches 3 to 10 the windows. Some batches are small, so the number of audit labels for a window of n rows is given by $\min(600, \lfloor n/2 \rfloor)$.

CLEAR-10. This image stream has ten object classes and a background class, spread over ten time-ordered buckets of about 3,300 images each [21]. Bucket 1 is the reference.

4.3 Injection benchmark

The type of a real drift is never known for certain, so the diagnosis is graded on drift that we inject ourselves. The benchmark uses the 2014 data of 5 states (CA, TX, NY, FL, WA) for both ACS tasks. Each state acts as a reference distribution with its own model, split as described above. For each of 10 seeds the split, the window and the injection are drawn again, which gives 50 replicates per configuration. A replicate draws a production window of 4,000 rows and a grading holdout of

2,000 rows from the test part. Each mechanism is injected into both at three intensities. The audit uses 600 labels. Proportions carry Wilson intervals [37] and means carry bootstrap intervals over replicates.

Design A injects the four types in ways that partly match how the tests work. Covariate shift rescales two numeric features (ACS Income: AGEP and WKHP) for a random subset of rows. Prior shift drops rows of one class until the positive rate rises by 0.14, 0.23 and 0.32. Concept drift inverts the labels of the 15, 30 and 50% of rows with the highest value of one feature (ACS Income: SCHL). Label noise flips a random 8, 18 and 32% of the labels. Each replicate also contains a control window without drift.

Design B uses mechanisms that the tests were not built for. Covariate shift becomes selection bias. Each row is kept with a probability given by $\sigma(\beta s(x))$, where σ is the logistic function, s is a seeded random score over all inputs and $\beta \in \{0.5, 1, 1.5\}$. The rows stay real and $P(Y \mid X)$ is exactly unchanged. Prior shift lowers the positive rate by 0.08, 0.14 and 0.20. Concept drift inverts the labels in the top 15, 30 and 50% of a second random score, which marks out an oblique region that no single feature describes. For ACS Income a second concept drift raises the income threshold to $55,000, $60,000 and $70,000. Label noise is class-conditional: positive labels are flipped to negative at rates of 10, 20 and 35%.

4.4 Baselines and ablations

A diagnosis is only worth running if it does better than simpler options. Every method below names one of the five outcomes (no drift or one of the four types) from the same tests and the same audit labels. Three rules stand for current practice. *Input monitoring* reads any input alarm as covariate shift, which is how drift alarms are commonly read. *BBSE first* reads a shift in the predicted labels as prior shift [1, 5] and any other alarm as covariate shift. It is our label-free stage without the consistency test. *Error-rate monitoring* also reads a significant drop in audit accuracy as concept drift, as an error-rate detector would [8].

The *learned meta-classifier* checks whether a learned rule could replace the tests. It is a GB model trained on nine statistics of each window: the evidence $-\log_{10} p$ of each block test, the share of features with residual shift, the BBSE prior change, the CBPE change, the evidence and size of the drop in audit accuracy and the evidence of the error structure test. In design A it is validated by leaving one state out. In design B it is trained on design A without the state being predicted, so it never sees a held-out mechanism. Three ablations take parts out of our procedure: the MMD block, the confidence term of the audit null and, in a third variant, the confidence bins, which are replaced by cells of predicted class and confidence.

Each method is compared with ours through the per-replicate difference in the share of correctly typed windows, tested by sign-flip permutation (10,000 flips) with Holm correction. Replicates are the unit of analysis because the windows of one replicate share a model and a reference.

5 Results

The benchmark comes first: how often the type is named, whether the procedure carries over to drift it was not designed for and how well its building blocks work. Real drift follows.

5.1 Drift type against baselines

The most direct test of the diagnosis is how often it names the injected type, compared with the baselines. Table 3 gives this share for every configuration, each with 50 replicates.

Table 3 Share of benchmark windows with the correct drift type (%), for each model family, task and design

	Inc. A	Inc. B	Emp. A	Emp. B
Logistic regression				
Ours	94.3	62.4	85.7	80.5
Input monitoring	30.3↓	20.0↓	30.5↓	25.0↓
BBSE first	30.5↓	27.7↓	30.5↓	39.2↓
Error-rate monitoring	52.3↓	48.5↓	52.2↓	62.8↓
Meta-classifier	97.5	31.1↓	95.8↑	48.2↓
Ours, no MMD	95.2	62.9	86.6	81.3
Ours, no confidence	79.1↓	62.9	83.7↓	80.7
Ours, class cells	93.1	65.6↑	86.2	91.8↑
Gradient boosting				
Ours	94.2	63.2	90.5	81.5
Input monitoring	30.3↓	20.0↓	30.5↓	25.0↓
BBSE first	30.5↓	28.1↓	30.6↓	40.3↓
Error-rate monitoring	52.5↓	49.6↓	53.1↓	64.8↓
Meta-classifier	98.5↑	39.2↓	97.4↑	51.2↓
Ours, no MMD	95.1	63.9	90.5	81.3
Ours, no confidence	78.0↓	62.9	85.4↓	82.0
Ours, class cells	92.9↓	66.5↑	90.6	93.7↑

Inc. and Emp. stand for ACS Income and ACS Employment. A and B are the two designs. An arrow marks a method that differs significantly from ours after Holm correction: a down arrow means that ours is better and an up arrow that ours is worse. Within each model

family the upper block lists the competing methods and the lower block the ablations of our procedure.

With LR on ACS Income, our procedure names the type in 94.3% of design A windows (95% interval [92.3, 95.8]). The best rule baseline reaches 52.3%. The rules are held back by design, since each can name only some of the five outcomes. Over all eight configurations our procedure is significantly better in 28 of the 32 comparisons with a baseline or the learned rule. It is significantly worse in 3, all against the learned meta-classifier on design A (ACS Employment LR, ACS Income GB and ACS Employment GB).

On design A the learned meta-classifier is the strongest competitor. Validated on states it has not seen, it reaches 97.5% on ACS Income, above our 94.3%. The held-out mechanisms of design B turn this around. Trained on design A, the meta-classifier drops to 31.1% on ACS Income and 48.2% on ACS Employment, while our procedure keeps 62.4% and 80.5%. The learned rule had tied each type to the size and shape of the injected shift. It had learned, for example, that covariate shift means a large shift in two features. It therefore misses selection bias, which moves many features a little. This is the main argument for tests over a learned rule. The tests encode what the drift types are, not what the training examples happened to look like.

The overall rates do not show which types get confused with which. Fig. 3 breaks the results down by type for LR.

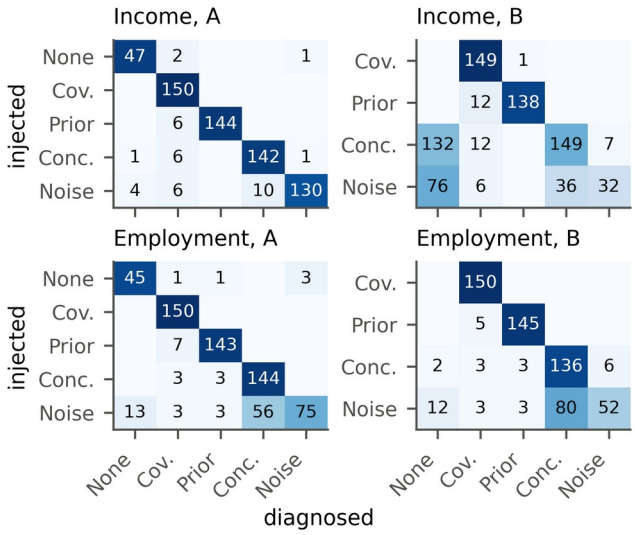


Fig. 3 Injected (rows) and diagnosed (columns) drift types on the benchmark with LR, in counts of windows; darker cells hold a larger share of their row

A perfect diagnosis would fill only the diagonal. Design B has no control windows, so false alarms are measured on design A. There, most errors sit in the label noise row, where noise is read either as concept drift or as no drift. The control row holds the false alarms. Our procedure raised one on 13 of 200 control windows (6.5%), 7 in the label-free stage and 6 in the audit. That is below the bound of 10% derived in Section 3.3. In design B the errors gather in the concept and noise rows, for reasons given in Section 5.2.

The ablations show which parts of the procedure carry the weight. Without the confidence term in the audit null, accuracy on ACS Income design A falls by 15.2 points, because uniform label noise then looks like concept drift. Dropping the MMD block changes little on the benchmark (−0.9 points), whose shifts are visible feature by feature. Its value shows on the image embeddings of Section 5.4. Replacing the confidence bins by cells of predicted class and confidence helps on ACS Employment design B (91.8% against 80.5%) but costs accuracy on ACS Income design A (93.1%). Switching to it now would mean choosing the method after seeing design B. It therefore stays an ablation and a direction for future work.

5.2 Held-out mechanisms

Design B asks whether the procedure carries over to drift it was not built for. It is the harder of the two tests, since none of its mechanisms shaped any part of the procedure. Table 4 gives the results for each mechanism.

Table 4 Design B results by drift mechanism

	ACS Income				ACS Employment			
	LR		GB		LR		GB	
Mechanism	Corr.	Drop	Corr.	Drop	Corr.	Drop	Corr.	Drop
Covariate: selection bias	99	0.2	99	0.3	100	−0.3	100	−0.3
Prior: decrease	92	−2.8	93	−2.4	97	2.5	97	1.9
Concept: oblique region	95	21.9	95	22.5	91	15.7	97	17.7
Concept: income threshold	4	−0.3	6	0.2	–	–	–	–
Noise: class-conditional	21	2.8	23	3.1	35	7.7	33	8.0

Corr. is the share of windows typed correctly (%) and Drop is the mean true accuracy drop in points; a negative value means that accuracy rose. The income threshold mechanism exists only for ACS Income.

Three of the five mechanisms carry over well. Selection bias is named covariate shift in 99% of ACS Income windows with LR, even though no value is distorted and the shift is spread across all features. A lower positive rate is named prior shift in 92%. Concept drift in an oblique

region, which no single feature bounds, is still recognized in 95%.

The other two mechanisms are harder. A higher threshold relabels some rows in each window, yet model accuracy hardly moves (mean drop −0.3 points). With almost nothing lost, the audit stops at its first question and reports no drift in 132 of the 150 windows. This is intended, as the procedure looks for drift that costs accuracy and this drift costs almost none. Class-conditional label noise is recognized in only 21% of ACS Income windows, for two reasons. At low flip rates the loss of accuracy is too small to show up in 600 labels. At high rates the errors follow the true class, which is itself correlated with the features, so the error structure test reads the pattern as concept drift. A null model built on confidence alone cannot separate the two, a true limitation. Neither failure involves the label-free stage.

Results on ACS Employment and with GB point the same way. On design A the procedure reaches 85.7% on ACS Employment with LR, while GB brings it to 94.2% on ACS Income and 90.5% on ACS Employment. Label noise stays the hardest type there (50.0% of windows). The ranking of the methods survives the switch of model family, as expected for tests that see only the outputs of the model and where it errs.

5.3 Detection and accuracy estimation

The diagnosis rests on the detectors and on the accuracy estimate, so we also check both on the benchmark. Table 5 gives the alarm rate of each detector for each injected type, pooled over the four design A configurations.

Table 5 Detector alarm rates on design A by injected type

	Alarm rate (%) by injected type				
Detector	No drift [95% CI]	Covariate	Prior	Concept	Noise
KS	1.0 [0.3, 3.6]	100.0	100.0	1.0	1.0
PSI	0.0 [0.0, 1.9]	99.7	0.3	0.0	0.0
MMD	7.0 [4.2, 11.4]	100.0	100.0	7.0	7.0
Confidence	5.0 [2.7, 9.0]	93.8	86.7	5.0	5.0
BBSD	2.5 [1.1, 5.7]	100.0	100.0	2.5	2.5
Domain classifier	6.0 [3.5, 10.2]	100.0	72.0	6.0	6.0

The no drift column is the false alarm rate with its 95% confidence interval (CI). Over 200 control windows every interval contains $\alpha = 0.05$ or lies below it. All detectors catch covariate shift and all except PSI also catch most prior shifts. In concept drift and label noise windows the inputs are the same as in the control window of their replicate, so no detector can see them. This is why the audit is needed. Table 6 turns to the four accuracy estimates and compares them by their mean absolute error against the measured accuracy.

Table 6 Mean absolute error of the label-free accuracy estimates (points)

	Benchmark (design A)				Real windows			
Estimator	None	Cov.	Prior	Conc., noise	Inc.	Emp.	Gas	CLEAR
CBPE	**0.50**	**9.20**	3.15	14.71	1.86	1.06	30.98	0.82
AC	0.83	9.58	3.65	**14.66**	1.87	1.25	30.36	5.32
DoC	**0.50**	9.63	3.12	14.71	1.77	1.06	29.78	0.76
ATC	0.63	10.31	**2.43**	**14.66**	**1.15**	**1.02**	**26.95**	**0.62**

Cov. and Conc. stand for covariate shift and concept drift, Inc. and Emp. for the two ACS tasks. The smallest error in each column is in bold. On design A all four estimates stay within 0.83 points without drift and within 10.31 points under covariate shift. Under concept drift and label noise every error equals the whole loss, about 14.71 points, because the inputs do not change. Across these 8 comparisons ATC has the smallest error in 6 and CBPE in only 2. On the gas sensor data all four miss by 26.95 to 30.98 points on average. In batch 8 the mean confidence of the LR model falls from 0.973 to 0.595 and its accuracy to 0.330, yet CBPE estimates 0.618. The estimate is best read as an upper bound on accuracy.

5.4 Real drift

On real data the type of drift is not known, but its causes are on record. Here we ask whether the verdicts agree with those causes and whether the control windows stay quiet. Fig. 4 shows how accuracy changes over time in all four datasets.

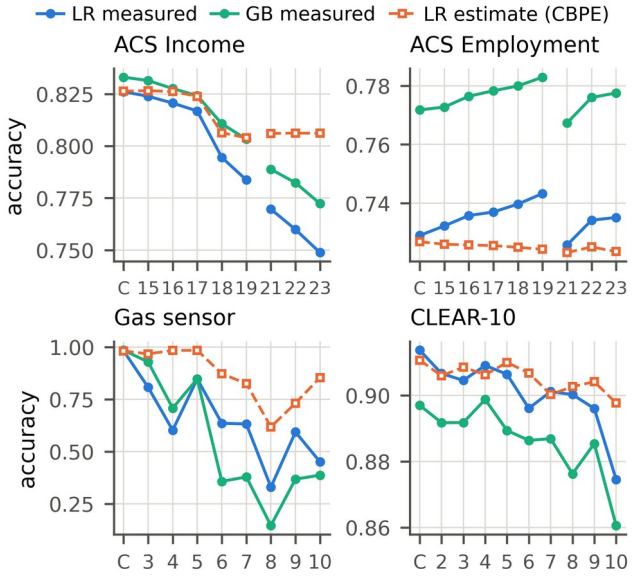


Fig. 4 Measured accuracy of both models and the label-free CBPE estimate on the temporal windows

C marks the control window. The numbers mark years (ACS), batches (gas sensor) or buckets (CLEAR-10). The gap in the ACS lines is the missing 2020 release.

ACS Income shows the clearest trend, so we start there. Its accuracy falls over time for both models, with LR going from 0.824 in 2015 to 0.749 in 2023, while the CBPE estimate falls far less. Table 7 lists the details behind the ACS Income panel of Fig. 4.

Table 7 Detection and diagnosis on the ACS Income windows. The alarms are those of the LR model: K is the KS test, M the MMD test, C the confidence test, B BBSD and D the domain classifier

	Accuracy			Alarms						
Window	ΔLR	ΔGB	Est. err.	K	M	C	B	D	Top shifts	Type
Control	0.0	0.0	0.0	–	–	–	–	–	–	–
2015	−0.2	−0.2	+0.3	√	–	–	–	–	OCCP	–
2016	−0.5	−0.5	+0.6	√	–	–	√	√	OCCP, POBP	Cov.
2017	−0.9	−0.9	+0.7	√	√	√	√	–	OCCP, POBP	Cov.
2018	−3.2	−2.2	+1.2	√	√	√	√	√	OCCP, POBP	Cov.
2019	−4.2	−3.0	+2.0	√	√	√	√	√	OCCP, RELP	Cov.
2021	−5.6	−4.4	+3.6	√	√	√	√	√	OCCP, RAC1P	Cov.
2022	−6.6	−5.1	+4.6	√	√	√	√	√	OCCP, RAC1P	Cov.
2023	−7.7	−6.1	+5.7	√	√	√	√	√	OCCP, RAC1P	Cov.
TX	−2.4	−2.7	+1.1	√	√	√	√	√	POBP, RAC1P	Cov.
NY	−2.2	−2.7	+0.8	√	√	√	√	√	POBP, RAC1P	Cov.
FL	−4.0	−4.3	+2.2	√	√	√	√	√	POBP, RAC1P	Cov.
WA	−2.5	−2.5	+0.9	√	√	√	√	√	POBP, RAC1P	Cov.

The years are the California windows and TX, NY, FL and WA the 2014 windows of the other states. Δ is the change in accuracy, in points, against each model's accuracy on the control window. Est. err. is the error of the CBPE estimate. A tick marks an alarm. Type is the verdict of the diagnosis, the same for LR and GB in every window (Cov. is covariate shift). Top shifts lists the two features with the largest effect size: occupation (OCCP), place of birth (POBP), relationship (RELP) and race (RAC1P).

Neither model raises an alarm on the control window. Both call CA 2015, whose accuracy did not change significantly, no drift and every other window covariate shift. The documented events can be read off the localization. The 2018 revision of the occupation codes left 26.6% of rows with a code that never occurred in 2014. Occupation lost weight in the model as a result (SHAP change −0.032). Relationship moves in 2019 (effect size 0.091), when its survey question was redesigned. Race moves from 2021 (effect size 0.186), once the Census Bureau had changed its coding of written race answers [34]. Wage growth, measured against a fixed income threshold, shows up in the labels instead: their positive share rises from 0.355 in the control window to 0.498 in 2023 while the model predicts only 0.338. Part of the 2023 loss comes from a change in what the labels mean, something no label-free test can detect.

The cost of a drift differs widely between datasets. On ACS Employment every window is diagnosed as covariate shift, yet LR accuracy never changes by more than 2.2 points; relationship (2019) and race (from 2021) shift there as well while the model barely responds. Here an alarm alone would overstate the problem; the accuracy estimate and the audit keep it in proportion. The gas sensor batches all come out as covariate shift too, as they should for aging sensors that alter $P(X \mid Y)$. The largest loss is in batch 8 (−65.2 points for LR). BBSE at first takes each gas batch for a prior shift, since the predicted labels move too. The consistency test corrects this: the sensors respond differently to a gas mix that has not changed.

CLEAR-10 drifts more gently: buckets 2 and 3 show no drift for either model, while buckets 4, 5, 6, 7, 8, 9 and 10 show covariate shift. In buckets 4, 5, 6 and 9 no single embedding dimension shifts significantly with LR. Only the MMD test sees the change while the predicted labels stay put, which is why the multivariate block is needed. The only detector alarm on any control window comes from the domain classifier baseline on CLEAR-10 ($p = 0.034$), a false alarm of the kind that α allows.

5.5 Reliability and triage

The last two stages act on single predictions and on retraining. The fused risk score is judged by how well it ranks wrong predictions above right ones. Where the fusion is learned, on ACS and CLEAR-10, its area under the receiver operating characteristic curve (AUROC) differs from that of confidence alone by −0.012 to +0.001 over 72 windows and by −0.002 to +0.009 on the benchmark. On the gas sensor array the calibration part holds too few wrong predictions (4 with LR and 11 with GB), so the risk falls back to confidence. Confidence alone carries most of the information about a single prediction.

For the triage, a retrain truly helps when importance weighting or the latest labeled window raises holdout accuracy significantly. Over 82 real settings (Table S1 in the Supplement), the triage is right in 54 of the 58 settings that have an earlier labeled window, because its recent-window probe is checked on real labels. Without such a window it relies on importance-weighted validation. It is then right in 23 of 24 settings, but mostly by advising against a retrain, since one truly helped in only 2 of them. On the benchmark its recall is 6%, because importance-

weighted validation cannot see gains outside the reference data. A dashboard presents all of these results step by step (Fig. S2 in the Supplement).

6 Discussion and conclusions

This paper showed that hypothesis tests with a stated error rate can name the drift type, which decides the right response, from label-free tests and a few hundred late labels. On injected drift the procedure beats rules from monitoring practice and the label shift literature and works for both model families. Unlike a learned meta-classifier, it also carries over to held-out drift, because the tests encode the definitions of the types. On real drift it reported no drift on any control window and traced documented events, from new occupation codes to aging sensors.

Several limits apply. Each benchmark window holds a single type, while real windows such as ACS Income 2023 mix several. The four types are also an idealization: sensor aging changes $P(X \mid Y)$ and, through it, $P(Y \mid X)$, yet our procedure reports it as covariate shift. Design B reduces, but cannot remove, the risk that the benchmark favors the tests. The weighted residual tests do not account for the sampling error of the BBSE weights. At several hundred thousand rows almost any shift is significant, so each decision carries an effect size. The audit lets through drift that does not hurt accuracy and often mistakes class-conditional label noise for concept drift. Deep models trained end to end, text and streaming data remain untested. The next steps are an audit conditioned on the full model output, a model of the sampling error of the BBSE weights and benchmark windows that mix several types.

References

[1] Rabanser, S., Günnemann, S., Lipton, Z. C. "Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift", In: Advances in Neural Information Processing Systems 32, Vancouver, Canada, 2019, pp. 1394–1406. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2019/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html [Accessed: 21 October 2026]

[2] Garg, S., Balakrishnan, S., Lipton, Z. C., Neyshabur, B., Sedghi, H. "Leveraging Unlabeled Data to Predict Out-of-Distribution Performance", In: The Tenth International Conference on Learning Representations, Virtual, 2022. [online] Available at: https://arxiv.org/abs/2201.04234 [Accessed: 21 October 2026]

[3] Kivimäki, J., Białek, J., Kuberski, W., Nurminen, J. K. "Performance Estimation in Binary Classification Using Calibrated Confidence", Machine Learning, 115(3), 67, 2026. https://doi.org/10.1007/s10994-025-06970-3

[4] Moreno-Torres, J. G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N. V., Herrera, F. "A Unifying View on Dataset Shift in Classification", Pattern Recognition, 45(1), pp. 521–530, 2012. https://doi.org/10.1016/j.patcog.2011.06.019

[5] Lipton, Z. C., Wang, Y.-X., Smola, A. J. "Detecting and Correcting for Label Shift with Black Box Predictors", In: Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, 2018, pp. 3122–3130. [online] Available at: https://proceedings.mlr.press/v80/lipton18a.html [Accessed: 21 October 2026]

[6] Roschewitz, M., Mehta, R., Jones, C., Glocker, B. "Automatic Dataset Shift Identification to Support Safe Deployment of Medical Imaging AI", In: Medical Image Computing and Computer Assisted Intervention – MICCAI 2025, Daejeon, South Korea, 2025, pp. 67–76. https://doi.org/10.1007/978-3-032-04981-0_7

[7] Maia Polo, F., Izbicki, R., Lacerda, E. G., Ibieta-Jimenez, J. P., Vicente, R. "A Unified Framework for Dataset Shift Diagnostics", Information Sciences, 649, 119612, 2023. https://doi.org/10.1016/j.ins.2023.119612

[8] Gama, J., Medas, P., Castillo, G., Rodrigues, P. "Learning with Drift Detection", In: Advances in Artificial Intelligence – SBIA 2004, São Luís, Brazil, 2004, pp. 286–295. https://doi.org/10.1007/978-3-540-28645-5_29

[9] Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., Smola, A. "A Kernel Two-Sample Test", Journal of Machine Learning Research, 13, pp. 723–773, 2012. [online] Available at: https://jmlr.org/papers/v13/gretton12a.html [Accessed: 21 October 2026]

[10] Lopez-Paz, D., Oquab, M. "Revisiting Classifier Two-Sample Tests", In: 5th International Conference on Learning Representations, Toulon, France, 2017. [online] Available at: https://arxiv.org/abs/1610.06545 [Accessed: 21 October 2026]

[11] Hendrycks, D., Gimpel, K. "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks", In: 5th International Conference on Learning Representations, Toulon, France, 2017. [online] Available at: https://arxiv.org/abs/1610.02136 [Accessed: 21 October 2026]

[12] Guillory, D., Shankar, V., Ebrahimi, S., Darrell, T., Schmidt, L. "Predicting with Confidence on Unseen Distributions", In: 2021 IEEE/CVF International Conference on Computer Vision, Montreal, QC, Canada, 2021, pp. 1114–1124. https://doi.org/10.1109/ICCV48922.2021.00117

[13] Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., Snoek, J. "Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty under Dataset Shift", In: Advances in Neural Information Processing Systems 32, Vancouver, Canada, 2019, pp. 13991–14002. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html [Accessed: 21 October 2026]

[14] Lundberg, S. M., Lee, S.-I. "A Unified Approach to Interpreting Model Predictions", In: Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 2017, pp. 4765–4774. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html [Accessed: 21 October 2026]

[15] Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., Torralba, A. "Learning Deep Features for Discriminative Localization", In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 2016, pp. 2921–2929. https://doi.org/10.1109/CVPR.2016.319

[16] Liu, F. T., Ting, K. M., Zhou, Z.-H. "Isolation Forest", In: 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413–422. https://doi.org/10.1109/ICDM.2008.17

[17] Shimodaira, H. "Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function", Journal of Statistical Planning and Inference, 90(2), pp. 227–244, 2000. https://doi.org/10.1016/S0378-3758(00)00115-4

[18] Sugiyama, M., Krauledat, M., Müller, K.-R. "Covariate Shift Adaptation by Importance Weighted Cross Validation", Journal of Machine Learning Research, 8, pp. 985–1005, 2007. [online] Available at: https://jmlr.org/papers/v8/sugiyama07a.html [Accessed: 21 October 2026]

[19] Cortes, C., Mansour, Y., Mohri, M. "Learning Bounds for Importance Weighting", In: Advances in Neural Information Processing Systems 23, Vancouver, Canada, 2010, pp. 442–450. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2010/hash/59c33016884a62116be975a9bb8257e3-Abstract.html [Accessed: 21 October 2026]

[20] Ding, F., Hardt, M., Miller, J., Schmidt, L. "Retiring Adult: New Datasets for Fair Machine Learning", In: Advances in Neural Information Processing Systems 34, Virtual, 2021, pp. 6478–6490. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2021/hash/32e54441e6382a7fbacbbbaf3c450059-Abstract.html [Accessed: 21 October 2026]

[21] Lin, Z., Shi, J., Pathak, D., Ramanan, D. "The CLEAR Benchmark: Continual LEArning on Real-World Imagery", In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, Virtual, 2021. [online] Available at: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/2838023a778dfaecdc212708f721b788-Abstract-round2.html [Accessed: 21 October 2026]

[22] Holm, S. "A Simple Sequentially Rejective Multiple Test Procedure", Scandinavian Journal of Statistics, 6(2), pp. 65–70, 1979. [online] Available at: https://www.jstor.org/stable/4615733 [Accessed: 21 October 2026]

[23] He, K., Zhang, X., Ren, S., Sun, J. "Deep Residual Learning for Image Recognition", In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 2016, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

[24] Siddiqi, N. "Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring", Wiley, Hoboken, NJ, USA, 2006. ISBN 978-0-471-75451-0 https://doi.org/10.1002/9781119201731

[25] Cochran, W. G. "Some Methods for Strengthening the Common χ^2 Tests", Biometrics, 10(4), pp. 417–451, 1954. https://doi.org/10.2307/3001616

[26] Phipson, B., Smyth, G. K. "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn", Statistical Applications in Genetics and Molecular Biology, 9(1), 39, 2010. https://doi.org/10.2202/1544-6115.1585

[27] Kish, L. "Survey Sampling", Wiley, New York, NY, USA, 1965.

[28] Peduzzi, P., Concato, J., Kemper, E., Holford, T. R., Feinstein, A. R. "A Simulation Study of the Number of Events per Variable in Logistic Regression Analysis", Journal of Clinical Epidemiology, 49(12), pp. 1373–1379, 1996. https://doi.org/10.1016/S0895-4356(96)00236-3

[29] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É. "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, 12, pp. 2825–2830, 2011. [online] Available at: https://jmlr.org/papers/v12/pedregosa11a.html [Accessed: 21 October 2026]

[30] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", In: Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 2017, pp. 3146–3154. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html [Accessed: 21 October 2026]

[31] U.S. Census Bureau "2020 ACS 1-Year Experimental Data Release", [online] Available at: https://www.census.gov/programs-surveys/acs/data/experimental-data.html [Accessed: 21 October 2026]

[32] U.S. Census Bureau "American Community Survey 2019 PUMS Data Dictionary", [online] Available at: https://www2.census.gov/programs-surveys/acs/tech_docs/pums/data_dict/PUMS_Data_Dictionary_2019.csv [Accessed: 21 October 2026]

[33] Beckhusen, J. B. "Recent Changes in the Census Industry and Occupation Classification Systems", U.S. Census Bureau, Suitland, MD, USA, Rep. ACS-TP78, 2020. [online] Available at: https://www.census.gov/library/publications/2020/demo/acs-tp78.html [Accessed: 21 October 2026]

[34] U.S. Census Bureau "Improvements to the Race Question (ACS User Note 2021-03)", [online] Available at: https://www.census.gov/programs-surveys/acs/technical-documentation/user-notes/2021-03.html [Accessed: 21 October 2026]

[35] Vergara, A., Vembu, S., Ayhan, T., Ryan, M. A., Homer, M. L., Huerta, R. "Chemical Gas Sensor Drift Compensation Using Classifier Ensembles", Sensors and Actuators B: Chemical, 166–167, pp. 320–329, 2012. https://doi.org/10.1016/j.snb.2012.01.074

[36] Rodriguez-Lujan, I., Fonollosa, J., Vergara, A., Homer, M., Huerta, R. "On the Calibration of Sensor Arrays for Pattern Recognition Using the Minimal Number of Experiments", Chemometrics and Intelligent Laboratory Systems, 130, pp. 123–134, 2014. https://doi.org/10.1016/j.chemolab.2013.10.012

[37] Wilson, E. B. "Probable Inference, the Law of Succession, and Statistical Inference", Journal of the American Statistical Association, 22(158), pp. 209–212, 1927. https://doi.org/10.1080/01621459.1927.10502953